

PDF 1

20 September 2026

12:41

-  Business KPIs i have build
-  Total Users
-  New Users by Month
-  Daily Active Users (DAU)
-  Monthly Active Users (MAU)
-  Total Posts
-  Posts by Content Type
-  Total Views
-  Average Views per Post
-  Total Likes
-  Total Comments
-  Total Shares
-  Engagement Rate
-  Average Engagement per User
-  Top 10 Creators
-  Most Viewed Posts
-  Most Liked Posts
-  Most Shared Posts
-  User Retention Rate
-  Content Performance by Type
-  City-wise User Distribution
-  Posting Trend by Month
-  Viral Content Analysis
-  Creator Growth Analysis
-  Platform Growth Dashboard
-  Executive Social Media Dashboard

-- Basic KPIs Level 1

-- Total users

```
select count(user_id) as total_users
from users;
```

-- Total Posts

```
select count(Post_id) as total_posts
from posts;
```

-- Total views

```
select sum(views) as total_views
from posts;
```

-- Total likes

```
select sum(likes) as total_likes
from engagement;
```

```

-- Total comments
select sum(comments) as total_comments
from engagement;

-- Total shares
select sum(shares) as total_shares
from engagement;

-- Average views per post
SELECT AVG(views) AS average_views_per_post
FROM posts;

-- level 2 content & user analysis

-- post by content type
select content_type , count(post_id) as post_by_content_type
from posts
group by content_type;

-- views by content type
select content_type, sum(views) as Total_views_By_content
from posts
group by content_type;

-- engagement by content type
select p.content_type , sum(e.likes + e.comments + e.shares) as engagement_by_content
from posts as p
join engagement as e
on e.post_id = p.post_id
group by p.content_type;

-- city wise user distribution
select city , count(user_id) as " city-wise user distribution"
from users
group by city;

-- Posts created by each user
select u.user_id , count(p.post_id) as "post created by each user"
from users as u
join posts as p
on p.user_id = u.user_id
group by u.user_id;

-- Average views per creator
select u.user_name, avg(p.views) as average_views_per_creator
from users as u
join posts as p
on p.user_id = u.user_id
group by u.user_name ;

-- Level 3: Engagement analysis
-- engagement rate for each post
select p.post_id,e.likes,e.comments,e.shares , p.views,((e.likes + e.comments + e.shares)/p.views*100)

```

```
as engagement_rate
from posts as p
join engagement as e
on p.post_id = e.post_id;
```

```
-- Most viewed posts
select post_id , views
from posts
order by views desc;
```

```
-- most liked posts
select post_id , likes
from engagement
order by likes desc;
```

```
-- most commented posts
select post_id , comments
from engagement
order by comments desc;
```

```
-- Most shareed posts
select post_id , shares
from engagement
order by shares desc;
```

```
-- posts with above average engagement
SELECT post_id,
       (likes + comments + shares) AS total_engagement
FROM engagement
WHERE (likes + comments + shares) >
      (
        SELECT AVG(likes + comments + shares)
        FROM engagement
      )
ORDER BY total_engagement DESC;
```

```
-- viral content analysis
with new_viral as(
select p.post_id,e.likes,e.comments,e.shares , p.views,((e.likes + e.comments + e.shares)/p.views*100)
as engagement_rate
from posts as p
join engagement as e
on p.post_id = e.post_id
)
select post_id , engagement_rate,
case
  when engagement_rate > 15 then "viral"
  else "normal"
end as viral
from new_viral;
```

```
-- Level 4: Advanced SQL
```

```

-- Rank creators by total engagement
with new_creators as(
select u.user_name , sum(e.likes+e.comments+e.shares) as total_engagement
from engagement as e
join posts as p
on p.post_id = e.post_id
join users as u
on u.user_id = p.user_id
group by u.user_name
)
SELECT user_name , total_engagement,
rank() over(order by total_engagement desc) as engagement_rank
from new_creators;

```

```

-- Rank posts by views

```

```

select post_id , views,
rank() over ( order by views desc) as NUm_of_views
from posts;

```

```

-- top 3 most-viewed posts
with top_3 as(
select post_id , views,
rank() over ( order by views desc) as top3_posts
from posts
)
select post_id , views, top3_posts
from top_3
where top3_posts <=3;

```

```

-- top 2 creators by total engagement
WITH creator_engagement AS (
    SELECT
        u.user_name,
        SUM(e.likes + e.comments + e.shares) AS total_engagement
    FROM users u
    JOIN posts p
        ON u.user_id = p.user_id
    JOIN engagement e
        ON p.post_id = e.post_id
    GROUP BY u.user_id, u.user_name
),
ranked_creators AS (
    SELECT
        user_name,
        total_engagement,
        RANK() OVER (ORDER BY total_engagement DESC) AS engagement_rank
    FROM creator_engagement
)
SELECT
    user_name,
    total_engagement,
    engagement_rank

```

```

FROM ranked_creators
WHERE engagement_rank <= 2;

-- monthly posting trend:
-- find number of post created in each month
SELECT
    MONTH(post_date) AS month,
    COUNT(post_id) AS total_posts
FROM posts
GROUP BY MONTH(post_date);

SELECT
    DATE_FORMAT(post_date, '%Y-%m') AS month,
    COUNT(post_id) AS total_posts
FROM posts
GROUP BY DATE_FORMAT(post_date, '%Y-%m')
ORDER BY month;

-- monthly user growth
-- find how many users joined the platform in each month.alter
select date_format(join_date,"%Y-%m") as month, count(user_id) as new_users
from users
group by date_format(join_date,"%Y-%m")
order by month;

-- CTE based creator analysis
with Creator_analysis as(
select u.user_name, count(p.post_id) as total_posts, sum(p.views) as total_views , sum(e.likes +
e.comments + e.shares) as total_engagment
from users as u
join posts as p
on u.user_id = p.user_id
join engagement as e
on p.post_id = e.post_id
group by u.user_name
)
select * from creator_analysis;

-- Window-function based growth analysis
WITH monthly_users AS (
    SELECT
        DATE_FORMAT(join_date, '%Y-%m') AS month,
        COUNT(user_id) AS new_users
    FROM users
    GROUP BY DATE_FORMAT(join_date, '%Y-%m')
)
select month ,new_users ,
lag(new_users) over(order by month) as new_month
from monthly_users;

-- level 5 : Dashboard KPIs
-- Daily active users-- how many unique users were active on the platform on a particular day>?

```

```

SELECT post_date,
       COUNT(DISTINCT user_id) as DAU
FROM posts
GROUP BY post_date;

```

```

-- MAU - monthly active users -- how many unique users were active in each month
select date_format(post_date, "%Y-%m") as month, count( distinct user_id) as MAU
from posts
group by month
order by month;

```

```

-- Retention Rate:
SELECT
  COUNT(DISTINCT p.user_id) AS returning_users,
  COUNT(DISTINCT u.user_id) AS original_users,
  COUNT(DISTINCT p.user_id) / COUNT(DISTINCT u.user_id) * 100 AS retention_rate
FROM users u
JOIN posts p
  ON u.user_id = p.user_id;

```

```

-- creator growth
WITH first_post AS (
  SELECT
    user_id,
    MIN(post_date) AS first_post_date
  FROM posts
  GROUP BY user_id
)
SELECT
  DATE_FORMAT(first_post_date, '%Y-%m') AS month,
  COUNT(user_id) AS new_creators
FROM first_post
GROUP BY month
ORDER BY month;

```

```

-- platform growth
WITH monthly_users AS (
  SELECT
    DATE_FORMAT(join_date, '%Y-%m') AS month,
    COUNT(user_id) AS new_users
  FROM users
  GROUP BY DATE_FORMAT(join_date, '%Y-%m')
)
SELECT
  month,
  new_users,
  LAG(new_users) OVER (ORDER BY month) AS previous_month_users,
  new_users - LAG(new_users) OVER (ORDER BY month) AS growth
FROM monthly_users
ORDER BY month;

```

```
-- Executive dashboard query
SELECT
  (SELECT COUNT(*) FROM users) AS total_users,

  (SELECT COUNT(*) FROM posts) AS total_posts,

  (SELECT SUM(views) FROM posts) AS total_views,

  (SELECT SUM(likes) FROM engagement) AS total_likes,

  (SELECT SUM(comments) FROM engagement) AS total_comments,

  (SELECT SUM(shares) FROM engagement) AS total_shares,

  (SELECT COUNT(DISTINCT user_id) FROM posts) AS active_users,

  (SELECT AVG(views) FROM posts) AS avg_views_per_post,

  (SELECT AVG(likes + comments + shares)
   FROM engagement) AS avg_engagement_per_post;
```